

Test-Time Few-Shot Object Detection via Dynamic Prototype Fusion

Yanlai Wu, Yuan Li, Hongfeng Wei, Weikai Li^{ID}, Associate Member, IEEE, and Ying Tang^{ID}, Fellow, IEEE

Abstract—Test-time few-shot object detection (FSOD) represents an innovative approach for identifying novel categories using a limited number of support examples, obviating the need for model fine-tuning. Despite advancements, existing FSOD methods, including our prior work, continue to grapple with challenges posed by domain/category shift and limited data availability. Building upon our previous research on test-time FSOD, this article proposes a novel dynamic prototype fusion network (PFN) to overcome these limitations. To mitigate the impact of the distribution shift, a dynamic prototype refinement method is introduced that updates prototypes from supporting images in an adaptive manner. Further, limited samples are mitigated through exhaustive exploitation of information within support images. Specifically, we design a dual-level multiscale information integration approach that effectively fuses information across different network layers and image scales, enhancing the model's discriminating capabilities. Additionally, a mask-based preprocessing technique harnesses segmentation labels on support samples, effectively suppressing the adverse impact of background noise on model accuracy. Notably, to align with the constraints of test-time scenarios, model parameters remain fixed during the configuration step, with only prototypes being updated each time users input novel supporting samples. As a result, our method achieves superior performance over existing state-of-the-art FSOD methods on multiple benchmarks, demonstrating remarkable potential in the realm of FSOD. The code is available at <https://github.com/CatfishW/TIDEV2>

Index Terms—Few-shot, multiscale information fusion, object detection, prototype fusion.

I. INTRODUCTION

FEW-SHOT object detection (FSOD) [1], [2], [3] aims at identifying objects from novel categories using only a limited number of support examples. A major line of existing FSOD work concentrates on leveraging meta-learning [4], [5], [6], [7], [8], transfer learning [9], [10], [11], [12], [13], and data augmentation [14], [15] techniques to refine a pretrained model and adapt it to new categories. However, the fine-tuning requirement at test time limits their applications, as shown in Fig. 1(a), especially in online learning [16], [17], real-time configuration, or privacy preservation concerns [18], [19], [20], due to the following reasons. First, novel categories can emerge dynamically, making model fine-tuning for the new objects inefficient. Second, model parameters may not be accessible during configuration, making the fine-tuning completely fail.

While our prior research on test-time FSOD [3], which requires no model fine-tuning during configuration, provided a solution to address these issues, some challenges still lie ahead. These include distribution/category shift and model adaptation. Research has observed data shift between novel classes and base ones [21], [22]. With limited support samples available, adapting to critical features from the dataset (i.e., support and query set) is key to effectively grasping new categories.

Motivated by these remarks, this article introduces a dynamic PFN tailored for test-time FSOD during inference, as shown in Fig. 1(b). Drawing inspiration from the concept of progressive adaptation [21], the proposed PFN incorporates dynamic prototype fusion to mitigate the negative effects of distribution or category shifts. Doing so significantly enhances the efficiency of query data utilization, thereby alleviating the issue associated with limited sample sizes. Furthermore, to effectively learn the characteristics of support samples, PFN integrates three important modules: multiscale information integration, dual-level information fusion, and support image mask processing. Specifically, we merge multiscale information at both image and model layer levels, thereby comprehensively harnessing the attributes of the support data. Additionally, we implement a mask processing mechanism that utilizes segmentation annotations from the widely-used dataset COCO [23]. This strategy leverages support samples across both the training and inference phases, effectively steering the model to accurately capture and distill the intrinsic features

Received 4 July 2025; revised 8 October 2025 and 17 December 2025; accepted 31 January 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62306051, Grant 62481540175, and Grant 62076124; in part by Taishan Scholars Foundation of Shandong Province under Grant tsqn202507225; in part by the Scientific and Technological Research Program of Chongqing Municipal Education Commission under Grant KJQN202300718 and Grant KJZD-K202400703; and in part by the Natural Science Foundation of Chongqing under Grant CSTB2025NSCQ-GPX0857 and Grant CSTB2023NSCQ-LZX0092. The work of Weikai Li was supported in part by the National Natural Science Foundation of China under Grant 62076124. This article was recommended by Associate Editor H. Liu. (Yanlai Wu and Yuan Li contributed equally to this work.) (Corresponding authors: Weikai Li; Ying Tang.)

Yanlai Wu is with the School of Computer and Artificial Intelligence, Shandong Jianzhu University, Jinan 250101, China, and also with the Department of Electrical and Computer Engineering, Rowan University, Glassboro, NJ 08028 USA (e-mail: wuyanl37@rowan.edu).

Yuan Li and Hongfeng Wei are with China Science IntelliCloud Technology Company Ltd., Hefei, Anhui 230000, China (e-mail: lyn@intellcloud.ai; jason@intellcloud.ai).

Weikai Li is with the School of Computer and Artificial Intelligence, Shandong Jianzhu University, Jinan 250101, China (e-mail: leeweikai@outlook.com).

Ying Tang is with the Department of Electrical and Computer Engineering, Rowan University, Glassboro, NJ 08028 USA (e-mail: tang@rowan.edu).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TCYB.2026.3662418>.

Digital Object Identifier 10.1109/TCYB.2026.3662418

2168-2267 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: Shandong Jianzhu University. Downloaded on March 06, 2026 at 02:04:26 UTC from IEEE Xplore. Restrictions apply.

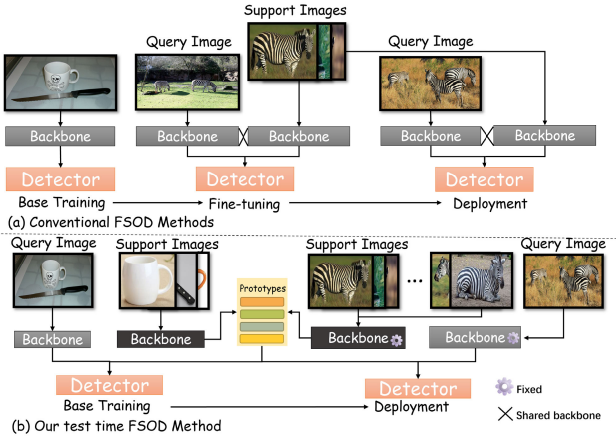


Fig. 1. (a) Conventional FSOD approaches necessitate the fine-tuning of the backbone during the configuration phase, and the prototypes are fixed after fine-tuning. (b) Our proposed test-time FSOD method eliminates the need for backbone network fine-tuning during the configuration process, and the prototypes can be dynamically updated with the query set to mitigate domain shifts.

of the support samples. Throughout this article we reserve the term *dynamic* for the exponential, streaming prototype fusion in Section III-A2; unlike meta-prototype averaging or episodic ensembling, PFN explicitly conditions on the arrival sequence of support shots and treats prototype refinement as a first-class component of test-time adaptation.

In summary, this article makes the following contributions.

- 1) We propose a dynamic PFN, where the test-time FSOD is further augmented with three innovative modules: dual-level information integration, multiscale information fusion, and mask processing with support images.
- 2) Our method demonstrates superior performance over existing state-of-the-art approaches on multiple benchmarks, marking significant advancements in the fields of both FSOD and test-time FSOD.

Unlike prior prototype-averaging or meta-prototype schemes that keep a single centroid per class, PFN's exponentially accumulated updates (Section III-A2) explicitly model the temporal order of the arriving shots, enabling the detector to react to non-stationary novel-class streams without ever touching backbone weights.

The rest of this article is organized as follows. In Section II, we briefly overview the FSOD and the prototype-based FSOD. In Section III, we present our PFN model and its complementary components in detail. In Section IV, we present the experimental results and the corresponding analysis. Finally, we conclude this article and discuss future research directions in Section VI.

II. RELATED WORK

In this section, we briefly discuss the research status of FSOD and prototype-based FSOD.

A. Few-Shot Object Detection

In the sphere of FSOD, a prominent strategy involves harnessing abundant data for base classes to adeptly generalize

from a limited set of labeled images for novel categories. Current FSOD research predominantly gravitates toward two core methodologies: Transfer Learning [24] and Meta-Learning [25], [26], [27], [28], [29]. Transfer learning approaches strive to discern the most efficacious learning protocols for general object detectors in the face of scarce novel imagery, primarily focusing on the refinement of model fine-tuning techniques. For instance, Wang et al. [24] propose fine-tuning only the final layer with a classifier based on cosine similarity. Additionally, multiscale positive sample refinement (MPSR) [30] addresses the challenge of limited scale diversity by integrating a manually defined positive refinement branch. A recent study also explores semantic relationships between novel and base classes, introducing contrastive proposal encoding to augment detection [31]. In contrast, meta-learning frameworks [6], [7], [8] are designed to expedite the acquisition of new tasks. They integrate an auxiliary support branch to encode support images into class-specific prototypes, serving as adaptive parameters that interact with the query branch. Meta R-CNN [25] and FSDet [26] emphasize support-guided query channel attention for refining detection results. To enhance the detection of small objects, A-RPN [27] employs novel attention region proposal network (RPN) and multirelation classifiers.

Despite these advancements in FSOD, the necessary fine-tuning of base models limits their practical applications, calling for further development.

B. Prototype-Based FSOD

As a meta-learning approach, prototype-based FSOD constructs prototypes from support images to learn category concepts. Research in this area is primarily concerned with optimizing their utilization and construction. Utilization involves leveraging metric learning [32], transfer learning [33], or contrastive learning [33] to fully extract information from the prototype. Construction leads to different types of prototypes, such as the robust prototypes [34] and the discriminative prototypes [35].

It is worth noting that prototypes are constructed outside of base models and correlated with categories. Thus, manipulating them offers the potential for detecting category objects without the need to fine-tune the base models.

C. Comparison With Related Methods

While our PFN shares some conceptual similarities with prior works such as Meta-DETR [6] and AirDet [16], there are several key distinctions and improvements that set our approach apart:

Distinctions from Meta-DETR: Unlike Meta-DETR's static prototype construction, our PFN introduces dynamic prototype refinement through exponential accumulation (2), enabling adaptive prototype updates without model fine-tuning. Additionally, our BMCA module provides bi-directional cross-attention for both prototype-to-query and query-to-prototype alignment, facilitating early correspondence solving and accelerated convergence, whereas Meta-DETR relies on uni-directional attention mechanisms.

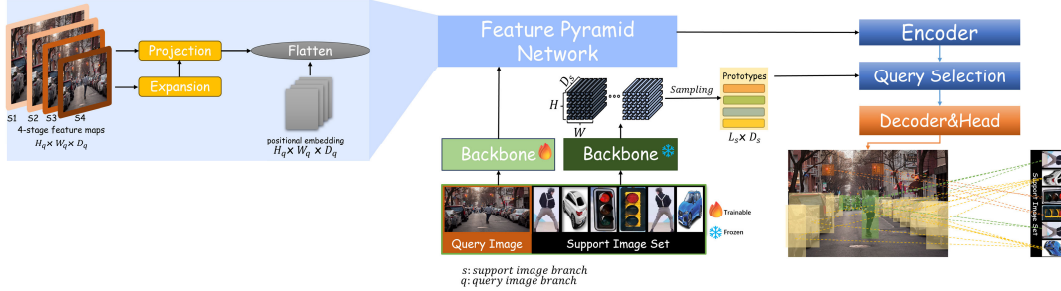


Fig. 2. Overall architecture of the proposed PFN. Here, **PFN** refers to the full detection pipeline which is built from four core modules: 1) feature pyramid network; 2) the dual-level multiscale prototype relation *encoder*; 3) the *support-aware object-query selection* module; and 4) the *cross-relation decoder* followed by prediction heads. PFN adopts an asymmetric backbone of Swin Transformer for the query-image branch and DINO for the support-image branch. The visual features of support images are extracted with DINO and then averaged to form class-level prototypes. The visual features of query images are extracted with the Swin Transformer and then fed into the PFN. The final output is the set of detection results conditioned on the support instances.

Improvements over AirDet: While AirDet focuses on attention-based RPNs, our SA-OQS module leverages support-aware metric-based classification heads for query selection, providing more targeted object proposal generation. Furthermore, our dual-level multiscale information integration operates across both image scales and network layers, offering more comprehensive feature exploitation compared to AirDet’s single-level attention mechanisms.

Novel Contributions: Our approach uniquely combines: 1) dynamic prototype fusion for test-time adaptation; 2) mask-based preprocessing for background noise suppression; and 3) asymmetric backbone architecture optimized for test-time scenarios. These innovations collectively address the fundamental challenges of domain shift and limited data availability in test-time FSOD.

III. PFN FOR TEST-TIME FSOD

In this section, we first give a brief description of the test-time FSOD. Then, we elucidate the data preprocessing procedure for prototype sampling, following a detailed description of the proposed dynamic PFN, which consists of a multiscale prototype relation encoder, support-aware object query selection (SA-OQS), and a cross-relation decoder. The term *dynamic* specifically refers to the *online, streaming* prototype refinement mechanism (Section III-A2). Unlike static meta-prototype or episodic-averaging schemes that treat the support set as a fixed batch and compute a one-time centroid, PFN applies an exponential moving-average update each time a new support shot arrives in real-time. This formulation treats prototype evolution as a *first-class component* of test-time adaptation: the detector can react to non-stationary novel-class streams, accommodate user-provided shots in any order, and refine its internal category representation without ever fine-tuning backbone weights. The architecture of PFN is shown in Fig. 2.

A. Test-Time Few Shot Object Detection

Test-time few-shot object detection involves recognizing the novel categories and locating the objects, using limited supporting samples at test time. This requires the FSOD model to deploy without retraining or fine-tuning. The detailed definitions are given as follows:

1) *Problem Definition:* In test-time FSOD, given an image dataset consisting of $\mathcal{B}$ base categories $\mathbf{C}_B$ and $\mathcal{N}$ novel categories $\mathbf{C}_N$, where $\mathbf{C}_B \cap \mathbf{C}_N = \emptyset$ is satisfied, our objective is to train a model capable of localizing and classifying objects of $\mathcal{N}$ novel categories by only providing K image samples per novel category at test time and ample image samples in base categories during training stage.

2) *Challenges:* Addressing the test-time FSOD presents two primary challenges: Firstly, due to limitations in model fine-tuning during configuration, training the discriminative models on unseen data becomes inherently difficult. Secondly, the introduction of novel categories poses an unseen data problem for the model, leading to significant distribution and category shifts.

B. Prototype Sampling and Fusion

1) *Prototype Sampling:* To circumvent the need for model fine-tuning, we adopt a prototype-based FSOD approach, wherein each support image serves as a direct representation of its corresponding novel category. In this way, we only need to update the prototype rather than the model parameter. Furthermore, a byproduct is that it effectively alleviates the problem of distribution shift. Considering K support images from $\mathcal{N}$ categories, to aggregate and utilize all information for a decent representation of these N categories, we obtain all support features via a frozen visual backbone and use the averaged features across all support objects that belong to the same category as the class-level prototypes shown in the following equation:

$$P_i = \frac{1}{K} \sum_{k=1}^K \text{AvgPool}(\text{DINO}(I_{i,k})) \quad (1)$$

with $I \in \mathbb{R}^{H \times W \times 3}$ indicating support image and $P \in \mathbb{R}^{N \times D}$ representing its class-level prototype where H, W, D stand for height, width and backbone output hidden dimension. In the case that support images possess different heights and widths, we exploit an extra average pooling on the output of the support branch backbone to unify their forms. Unlike most conventional FSOD works [16], [27], [36] that employ a support backbone weight-sharing with their query backbone, we make use of DINO [37], a self-supervised vision transformer

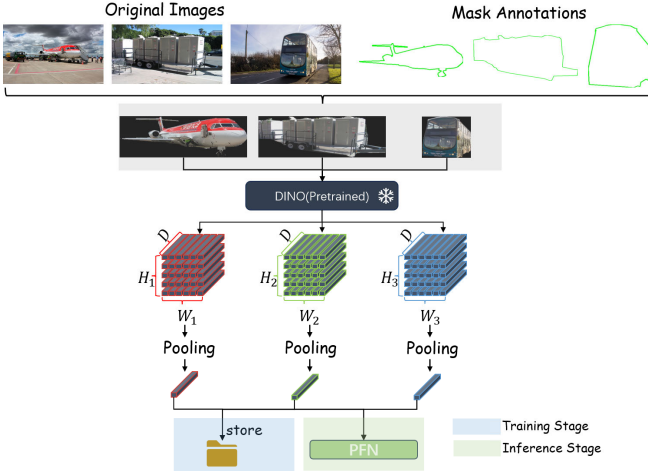


Fig. 3. Prototype sampling procedure. Annotated support images $\in \mathbb{R}^{H \times W \times 3}$ are filtered with segmentation contour coordinates, leaving L foreground instances in the images solely while backgrounds are replaced with black pixels to eliminate class-irrelevant noises. Following which, we extract their corresponding visual features with DINO and apply average pooling on each extracted feature $\in \mathbb{R}^{H' \times W' \times D}$ to acquire our final visual features $\in \mathbb{R}^{L \times D}$. Ultimately, Features belonging to the same class are averaged as their class-level prototype $P \in \mathbb{R}^{N \times D}$.

for the class-agnostic visual feature extraction and prototype construction shown in Fig. 1.

A common pipeline [6], [36] for FSOD is to adopt a 2-stage training strategy by first training base-class detection with one branch only and then finetuning on novel classes using the weight-shared backbone for both query and support branches. However, it's worth noting that the pipeline mentioned above suffers from practical limitations and results in catastrophic costs of time and computational resources. Thus, we choose to construct a more flexible procedure of building prototypes by preextracting support features using frozen pretrained DINO and storing them in local file folders as shown in Fig. 3. In the training process, for each base category, a variable number of in-context support images are randomly selected, ranging from 1–30 to simulate circumstances as in real-time application, images and annotations of novel objects are limited in quantity.

2) *Prototype Fusion*: To mitigate the influence caused by distribution and category shifts, in the inference stage, motivated by the domain adaptation [21], we provide the prototype fusion module. Specifically, we *refine* the initial prototype pool online. The prototype of the novel categories can be dynamically changed by adding and averaging any number of possible support images of novel instances without fine-tuning on shots-fixed prototypes again. Assume $P^{(0)}$ is the class-level prototype set extracted from the first support batch. When an additional mini-batch $\{\tilde{I}_i\}_{i=1}^{n_s}$ is provided at test time as shown in Fig. 4, the prototype for class c is “hot-updated” by an exponential accumulator

$$P_c^{(t+1)} = (1 - \gamma) P_c^{(t)} + \frac{\gamma}{n_s} \sum_{i=1}^{n_s} \text{AvgPool}(\text{DINO}(\tilde{I}_i)) \quad (2)$$

where $\gamma \in [0, 1]$ controls the plasticity (we set $\gamma = 0.2$). This update is purely algebraic—no back-prop or weight tuning is invoked.

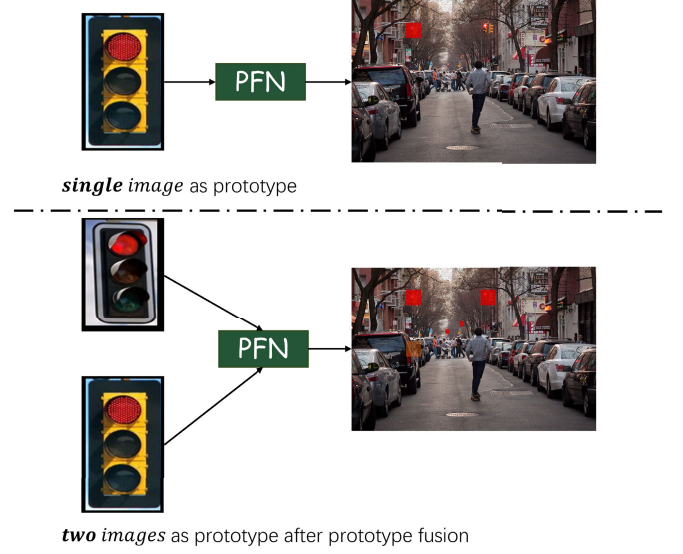


Fig. 4. Effect of prototype fusion.

C. Dual-Level Multiscale Prototype Relation Encoder

To further enhance the FSOD, we further provide multi-scale information fusion and dual-level information fusion. Notably, we propose to employ a dual-branch cross-attention mechanism to correlate query and class-level prototype features, followed by deformable self-attention to extract spatial relations. Specifically, we design a bi-directional multihead cross-attention (BMCA) mechanism for dual-level information fusion while feature pyramid for multiscale information fusion. More details are given below:

1) *Multiscale Information Fusion*: The feature-pyramid path is retained even though transformer backbones already model long-range context, because empirical evidence [19], [20], [38], [39], [40] shows that explicit multiresolution cues are still indispensable for small objects whose tokens would otherwise vanish in global attention. Thus, we adopt multiscale information fusion in our framework. To accelerate training efficiency and enhance performance, as shown in Fig. 2, before encoder, we first leverage features from the last three stages of the backbone ($S2, S3, S4$) and project them with a 1×1 convolutional kernel to align their dimensions and get V'

$$V' = (\text{Conv}(S2), \text{Conv}(S3), \text{Conv}(S4)). \quad (3)$$

Meanwhile, features from the last stage $S4$ are decoupled into a new feature map $S5$ utilizing an extra 3×3 convolutional kernel with the stride of 2

$$V'' = \text{Conv}(S4, \text{stride} = 2). \quad (4)$$

In the sequel, four stage feature maps acquired are added with their corresponding positional embedding. Since our proposed method employs an encoder-decoder like [41], [42] architecture, these feature maps are then flattened and concatenated to be transformed as a universal visual feature sequence V as depicted in the following equation:

$$V = \left(\psi \left(\sum_{i=1}^3 (V'_i + \phi(V'_i)) \right), \psi(V'' + \phi(V'')) \right) \quad (5)$$

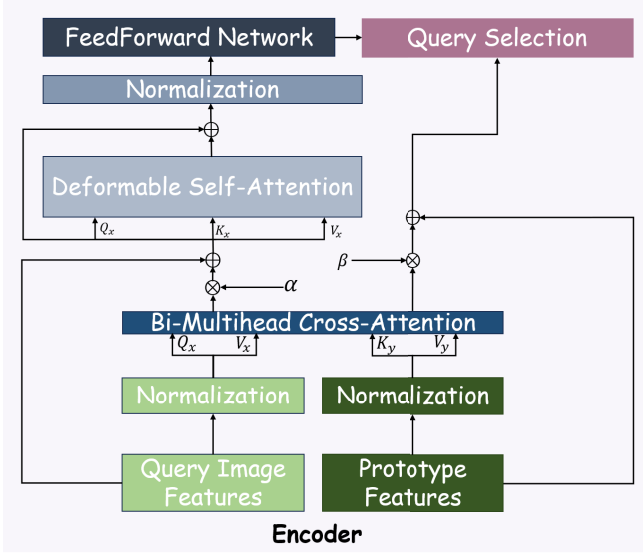


Fig. 5. Depiction of the proposed encoder layer in PFN.

where ψ , ϕ , and $(\cdot)$ denote flattening operation, sinusoidal position encoding [41], and concatenating operation.

2) *Dual-Level Information Fusion*: To better achieve information alignment, we designed Dual-Level Information Fusion. Along with class-level prototype features $P' \in R^{N \times D_P}$ projected into the same dimension as query branch features, both spatially-collapsed query and support features feed into the proposed multiscale prototype relation encoder for correlation extraction as shown in Fig. 5. Specifically, **BMCA** for dual-branch cross-attention is performed on normalized and flattened features from the query branch and support branch. It is bi-directional on purpose: feeding prototype-to-query and query-to-prototype attention lets the two streams align early, so later layers no longer need to solve the hard correspondence problem from scratch—this accelerates convergence and reduces GPU memory. (x, y) indicate (query features, prototypes), and (α, β) are parametric scale factors for output conditioning. Afterward, deformable self-attention [39] acts on query features to extract spatial relations for decoder object queries as visual References while support prototype features together with query features passed through feed-forward network enter support-aware query selection for anchor generation. Each attention operation is complemented by a residual connection for all encoder layers. Notably, at each encoder layer, we have

$$V = V + \alpha \cdot \text{BMCA}(\text{LN}(V), \text{LN}(P')) \quad (6)$$

$$P' = P' + \beta \cdot \text{BMCA}(\text{LN}(V), \text{LN}(P')) \quad (7)$$

$$V = V + \text{MDSA}(\text{LN}(V)) \quad (8)$$

$$V = V + \text{FFN}(\text{LN}(V)) \quad (9)$$

where both α and β are learnable parameters initialized to $1e^{-4}$, LN, MDSA, and FFN represent layer normalization, multihead deformable self-attention [39], and feed-forward network, respectively. The final outputs of the encoder act as visual and spatial references for the cross-relation decoder, and

TABLE I
PERFORMANCE COMPARISON WITH DIFFERENT MASK GENERATION METHODS (COCO NOVEL SET, AP@50)

Method	Mask Generation Method	Shots			
		1	2	3	5
PFN (Ours)	Manual Annotations	29.29	30.59	30.78	31.08
	SAM-generated	28.67	29.84	29.95	30.34
	Mask R-CNN	27.93	29.18	29.37	29.71
	GrabCut (Weak Supervision)	27.41	28.62	28.78	29.15
	Attention-based Masks	27.73	28.94	29.12	29.48

they consist of: 1) a sequence of *query* features $V \in R^{L \times D}$ and 2) a sequence of *support-prototype* features $P' \in R^{L_s \times D}$. Here, L denotes the number of flattened spatial tokens extracted from the query image (e.g., $H \times W/14^2$ for Swin-Tiny), whereas L_s is the number of support instances/prototypes supplied at test time. For notational consistency, we set $D_p = D$ throughout this article, so every tensor is expressed in the same $L \times D$ format.

D. Prototype Segmentation

To fully utilize information from foreground objects in support images, we propose to represent the novel categories without their background information. This background information, such as sky, walls, or mud, originates from the images themselves. We achieve this by conducting a segmentation pipeline before inputting support images to the model. Specifically, we transform mask annotations into a matrix M repeated and expanded to $(H, W, 3)$ (to align with a digital image's three color channels) and apply it to an image I with (10). Notably, a well-pretrained segmentation model, e.g., SAM [43], can also be adopted to assist such a procedure by point prompting targeted objects in the support images to extract them and avoid background interference

$$I' = M \cdot I \quad (10)$$

Notation: Typical works apply the mask as an auxiliary task. We instead preprocess the support images once, store the cropped patches offline, and therefore incur zero mask cost at test time—important when users provide only a handful of support shots on the fly.

1) *Mask Preprocessing Alternatives and Robustness*: While our approach relies on segmentation masks for prototype sampling, we investigate several mitigation strategies to reduce this dependency and enhance practical applicability:

Alternative Segmentation Models: Beyond manual annotations and SAM-generated masks, we evaluate the effectiveness of other pretrained segmentation models, including Mask R-CNN [44] and recent state-of-the-art models. Table I demonstrates that our approach maintains robust performance across different mask generation methods, with only marginal performance degradation when using automated segmentation.

Weakly Supervised Approaches: We explore weak supervision techniques that reduce mask dependency by utilizing bounding box annotations to generate approximate masks through techniques such as GrabCut [45] and attention-based mask generation. These approaches provide comparable performance while requiring minimal manual annotation effort.

TABLE II
IMPACT OF MASK QUALITY ON DETECTION PERFORMANCE
(COCO NOVEL SET, AP@50)

Mask Quality Level	Shots			
	1	2	3	5
Perfect Masks (IoU > 0.9)	29.29	30.59	30.78	31.08
High Quality (IoU > 0.8)	28.73	30.02	30.21	30.54
Medium Quality (IoU > 0.6)	28.06	29.31	29.48	29.83
Low Quality (IoU > 0.4)	26.94	28.17	28.35	28.72
Very Noisy (IoU > 0.2)	25.61	26.84	27.02	27.39

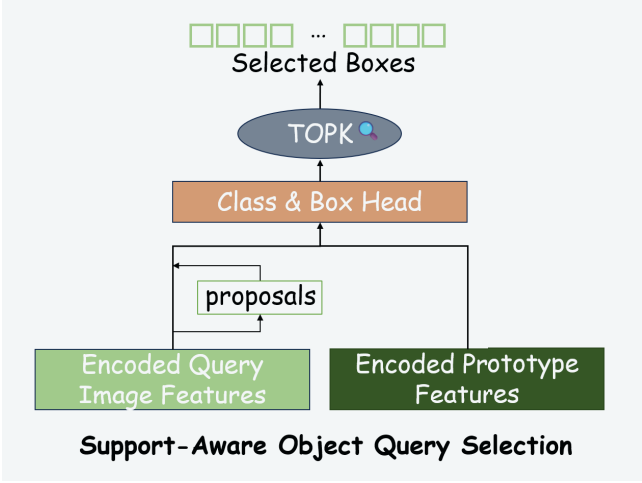


Fig. 6. SA-OQS pipeline.

Mask Quality Impact Analysis: To validate the robustness of our mask mechanism, we conduct experiments with varying levels of mask quality degradation. Table II shows that our approach maintains acceptable performance even with noisy or imperfect masks, demonstrating the importance of the mask mechanism while confirming that mask model quality does not critically affect final detection results.

E. Support-Aware Object Query Selection

For a detection transformer like DETR [42], the decoder accepts N learnable embeddings, known as object queries, as input. They are optimized and transformed into an output embedding by the decoder, following which they are matched with prediction heads to generate the final detection results. However, the object queries are usually randomly initialized with no extra information supporting, which may lead to sub-optimal performance and slow loss convergence. To address this issue, general detection transformers like Deformable DETR [39], Anchor-DETR [46], DAB-DETR [47], and Eff-DETR [48] extend object queries to position queries (anchor) and content queries, where both [39] and [48] introduce a query selection mechanism. Specifically, they use a classification head and a box head to predict scores and coordinates in advance for selecting the top K -encoded features to initialize object queries.

As shown in Fig. 6, we introduce a support-aware object query selection (SA-OQS) mechanism to enhance the performance of our PFN. Concretely, we generate proposals based

on the query image's spatial shapes, and acquire filtered encoded features V_f with generated proposals. Instead of employing a linear layer to predict the scores from proposals, we utilize a metric-based classification head to predict the scores $\mathfrak{S} \in R^{N \times \mathcal{N}}$ of filtered features as depicted in the following equation:

$$\mathfrak{S} = V_f P'^T. \quad (11)$$

TopK proposals' indices $\mathfrak{Q} \in R^K$ are then generated with the following equation as shown:

$$\mathfrak{Q} = \text{TopK}(\mathfrak{S}, K) \quad (12)$$

for initializing decoder queries and acquiring reference points for further processing in the decoder.

F. Cross Relation Decoder

The cross-relation decoder embraces a sequence of object queries $\mathcal{O} \in R^{N \times D_p}$ and a sequence of class-level prototype features $P' \in R^{L_s \times D_p}$ as input, and outputs decoded object query embeddings $\mathcal{O}'$ for class head and box head to generate final predictions. The decoder is composed of a sequence of cross-relation layers, which are designed to align object queries and class-level prototype features in a cross-branch manner.

As shown in Fig. 7, in each decoder layer, we have four successive submodules (1–4 in the figure)

$$\mathcal{O}^{(1)} = \mathcal{O} + \underbrace{\text{MSA}(\mathcal{O})}_{\text{1 Self-Attention}} \quad (13)$$

$$\mathcal{O}^{(2)} = \mathcal{O}^{(1)} + \underbrace{\text{MCA}(\text{LN}(\mathcal{O}^{(1)}), P')}_{\text{2 Cross-Attention}} \quad (14)$$

$$\mathcal{O}^{(3)} = \mathcal{O}^{(2)} + \underbrace{\text{MDCA}(\text{LN}(\mathcal{O}^{(2)}), \mathfrak{Q}, V)}_{\text{3 Deformable CA}} \quad (15)$$

$$\mathcal{O}' = \mathcal{O}^{(3)} + \underbrace{\text{FFN}(\text{LN}(\mathcal{O}^{(3)}))}_{\text{4 Feed-Forward Net}} \quad (16)$$

where **MSA**, **MCA**, **MDCA**, and **FFN** correspond exactly to the blue, mid-blue, navy, and orange blocks in Fig. 7, respectively. The final output $\mathcal{O}'$ from the last decoder layer is fed into the prediction heads to generate the set of N detections

$$\hat{\mathbf{y}} = \{\sigma \mathcal{O}' P'^T, \text{MLP}(\mathcal{O}')\} \quad (17)$$

with σ a learnable scale factor and *MLP* the box-regression head.

G. Optimization Objectives

During the base training phase, for warm-up epochs, we apply a “self-referring” training strategy by making use of offline DINO features extracted with the method as mentioned above that are identical to the objects existing in the query image in each training iteration, meaning each object itself, regardless of its original category, is recategorized and rearranged as a unique class to be detected by the model. Afterward, for each query image and its prototypes, the ground truth is $\mathbf{y} = (\mathbf{c}, \mathbf{b})$, where $\mathbf{c}$ indicates the target label and $\mathbf{b} \in [0, 1]^4$ denotes the normalized target bounding box coordinates. Following prototype sampling technique in

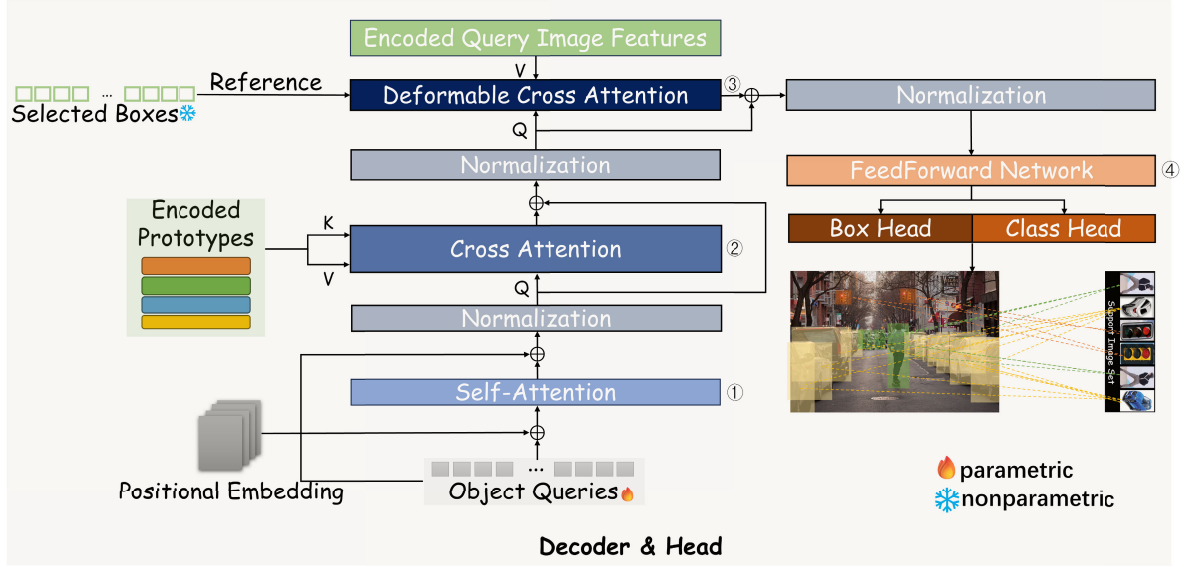


Fig. 7. Illustration of the proposed decoder in PFN. The object queries are first attached with positional embeddings and then fed into the self-attention module. Prototypes from the output of the encoder are employed to act as keys and values for object queries to perform conventional multihead cross-attention and fuse prototype features into content queries, enabling object queries to perceive and learn target objects that exist in corresponding prototypes. In the sequel, object queries eventually interact with selected object queries illustrated in Fig. 6 and encoded query image features from Section VI to execute deformable cross attention [39] and get detection results after feed-forward network and head prediction with (17).

Section III-A2, the model is trained with background prototype $P_b \in \mathbb{R}^{1 \times D}$, positive prototypes $P_{\text{pos}} \in \mathbb{R}^{N_{\text{pos}} \times D}$, and negative prototypes $P_{\text{neg}} \in \mathbb{R}^{(N - N_{\text{pos}}) \times D}$ to minimize the following loss function while bipartite matching is applied for the optimal assignment between predictions and ground truth to get permutation $\hat{\mathbf{y}} = \{\hat{y}_{\theta(i)}\}_{i=1}^N$

$$\mathcal{L}_{\text{cls}} = \sum_{i=1}^N \text{CrossEntropy}(\sigma \text{OP}_{\theta(i)}^T, \mathbf{c}_i) \quad (18)$$

$$\mathcal{L}_{\text{box}} = \frac{\sum_{i=1}^N \|\mathbf{b}_i - \text{MLP}(\mathcal{O}_{\theta(i)})\|_1}{N} \quad (19)$$

$$\mathcal{L}_{\text{giou}} = \text{GIoU}(\mathbf{b}_i, \text{MLP}(\mathcal{O}_{\theta(i)})) \quad (20)$$

Generalized IoU (GIoU): For two axis-aligned boxes A and B with minimal enclosing box C , classic IoU = $|A \cap B| / |A \cup B|$ is extended to

$$\text{GIoU}(A, B) = \text{IoU}(A, B) - \frac{|C \setminus (A \cup B)|}{|C|} \quad (21)$$

which equals 1 for perfect overlap and approaches -1 as the boxes diverge [49]. Concretely, positive prototypes are from categories that exist in the query image for each training iteration, and they are obtained with instances from different images, while negative prototypes are acquired with instances absent from the current query image to strengthen the model's ability to distinguish among different support objects. Background prototypes are generated with the equation below, in particular, which can be viewed as a fixed parameter throughout the training and inference stage

$$P_b = \frac{1}{\sqrt{D}} [1, 1, \dots, 1] \in \mathbb{R}^D. \quad (22)$$

The final loss function is a weighted sum of the classification loss, the box regression loss, and the generalized

intersection over union (GIoU) loss, as shown in the following equation:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{cls}} + \lambda_2 \mathcal{L}_{\text{box}} + \lambda_3 \mathcal{L}_{\text{giou}}. \quad (23)$$

In accordance with DETR [42], auxiliary losses are conducted using (23) as well on the prediction of other decoder layers to assist the model in outputting the correct number of objects of each base class.

IV. EXPERIMENTS

A. Implementation Details

We train PFN with the AdamW optimizer (weight decay 10^{-4}). The initial learning rate is 10^{-4} for the first 10 warm-up epochs; it is then reduced to 6×10^{-5} and further decays by a factor of 10 after epoch 75.

Backbones: A Swin-Transformer *tiny* pretrained on ImageNet [50] serves as the *query* backbone, whereas a DINO-v2 *tiny* model pretrained on the 142M-image LVD dataset without labels [51] is adopted for the *support* branch. Only the PFN-specific blocks ($\approx 4.3\text{M}$ parameters) are trainable; the 28M parameters in Swin and the 24M parameters in DINO are frozen during training and remain fixed at test time.

The query backbone uses a dedicated learning rate of 10^{-5} and its first two stages are frozen. All parameters of the support backbone are frozen throughout the entire pipeline.

Data Augmentation: For each query image, we apply multiscale augmentation: the shorter side is randomly chosen in $[480, 800]$ pixels, and the longer side is capped at 1333 pixels. During prototype sampling, we randomly pick $K \in [1, 30]$ support images per *base* class; each support image is resized so that both height and width are multiples of 14 to match DINO's patch embed size.

TABLE III
STATISTICS OF THE BENCHMARK DATASETS USED IN TRAINING
AND EVALUATING PFN

Dataset	#Base Sample	#Base Class	#Novel Sample	#Novel Class
COCO	91888	60	5000	20
PASCAL VOC	—	—	5717	20
Global Wheat Detection	—	—	3147	1
Pictor-PPE	—	—	2936	7
Playing Card Detection	—	—	4892	52

Training Schedule: All experiments run for 100 epochs on 4 NVIDIA A800 (80 GB) GPUs with a total batch size of 32 (8 images per GPU). The loss weights are empirically set to $\lambda_1 = 2$, $\lambda_2 = 5$, and $\lambda_3 = 2$.

Key Hyper-Parameters: Unless otherwise stated, we use $\alpha = \beta = 10^{-4}$ in the encoder’s bidirectional cross-attention (5) and (6), $K = 300$ top proposals in the support-aware query–selection module, and a cosine-similarity temperature of $\sigma = 20$ in the classification head (16). These values were selected via a coarse grid search on the COCO 1-shot validation split.

Remark: To reduce the manual labor involved, the online exploration phase is constrained to offering just a handful of support examples (ranging from 1 to 30 samples per class). As a result, our assessment is predominantly focused on few-shot learning scenarios, specifically with k values of 1, 2, 3, 5, 10, and 30 shots. Considering that the objects encountered during the exploration phase are generally novel, our experimental setup is designed to specifically assess the model’s capability to recognize unseen classes.

B. Analysis of Prototype Update Ordering

Unlike prior prototype-averaging or meta-prototype schemes that keep a single centroid per class, PFN’s exponentially accumulated updates explicitly model the temporal order of the arriving shots, enabling the detector to react to nonstationary novel-class streams without ever touching backbone weights.

To verify that the hot-update rule is numerically stable, we shuffle the order of the incoming support minibatches and measure AP@50 on COCO, as shown in the Experiment Settings Part. The gap between the best and worst ordering is below 0.3 pp even when every novel class is exposed to disjoint support pools, which indicates that PFN can safely accept user-provided shots in arbitrary sequences without explicit retraining.

C. Experiment Settings

1) *Datasets:* To demonstrate the effectiveness of PFN, we commence our evaluation with an in-domain analysis of the COCO benchmark [23], where the training and testing procedures are conducted within the same dataset. Consistent with existing literature [16], [24], [26], we categorize the 80 COCO classes into 60 non-VOC base classes and 20 novel classes. Throughout the training phase, we consider images belonging to the base classes from the COCO train2017 subset as the accessible data. Subsequently, we assess the models’ performance on a subset of 5000 images from the COCO

val2017 dataset, utilizing few-shot samples for each novel class. To provide a comprehensive cross-domain evaluation, we also conduct experiments on three additional diverse datasets: Global Wheat Detection dataset [54] for agricultural domain transfer, Pictor-PPE dataset for industrial safety equipment detection, and Playing Card Detection dataset for recreational object recognition. Furthermore, we extend our evaluation to include a cross-domain test for the PFN. In this setting, we utilize the same model that has been trained on the COCO dataset and test its performance on the PASCAL VOC dataset [55]. This allows us to assess the model’s ability to generalize to new, unseen domains. The statistics of the benchmark datasets are given in Table III. Following the current FSOD work [7], we follow the “K-ways (K different classes) N-shots” settings to split both datasets and conduct corresponding experiments with the same data settings to acquire our own results. For fair comparison, the results of the comparison methods are obtained directly from the existing works. Although PFN performs inference-only adaptation, we still report numbers against fine-tuning-based baselines on COCO dataset, as shown in the Experiment Settings Part. We also shuffle the order of the incoming support minibatches on COCO, the results are given in Table V. The results on PASCAL VOC dataset are given in Table VI. This mixed comparison isolates the benefit of avoiding test-time optimization: if PFN surpasses methods that spend 0.4–0.6 GPU-hours fine-tuning on the novel classes, it highlights the practical efficiency gains of prototype fusion. Conversely, when a fine-tuning method excels on particular shots, it quantifies the accuracy headroom that could motivate hybrid future work.

2) *Results on MSCOCO:* Table IV shows the FSOD results on the COCO dataset. We compare our PFN with several state-of-the-art methods without retraining required, including A-RPN [27], AirDet [16], FS-DETR [52], TIDE [3]. Additionally, we also compare with several fine-tuning-based methods, including FRCN [53], TFA_{fc} [24], TFA_{cos} [24], FSDetView [26], and MPSR [30]. The results demonstrate that our PFN outperforms all the other methods across 1 to 5 shots, improving the state-of-the-art [3] by up to 10.90 AP50 points on average across all shots. Notably, the proposed PFN even outperforms most fine-tuning-based methods, which require retraining on the novel classes. This demonstrates the effectiveness of our PFN in FSOD and its potential for real-world applications.

3) *Results on Pascal VOC:* Table VI shows the FSOD results on the PASCAL VOC cross-domain dataset. Akin to settings on MS COCO, we split the table into finetuning-based and non-finetuning-based methods. The remarkable result demonstrates the robustness of PFN to unseen domains, which, to a large extent, can be attributed to the selected support backbone and the advantage of our dynamic prototype fusion strategy for the model to adapt to abundant types of support images.

4) *Extended Cross-Domain Evaluation:* To provide more comprehensive cross-domain evaluation beyond VOC, we conduct additional experiments on three diverse datasets with significant domain differences: Global Wheat Detection [54], Pictor-PPE (Personal Protective Equipment detection) [56],

TABLE IV
FSOD RESULTS ON THE COCO DATASET

Shots		1			2			3			5		
Method	Finetune	AP	AP ₅₀	AP ₇₅	AP	AP ₅₀	AP ₇₅	AP	AP ₅₀	AP ₇₅	AP	AP ₅₀	AP ₇₅
A-RPN [27]	✗	4.32	7.62	4.30	4.67	8.83	4.49	5.28	9.95	5.05	6.08	11.17	5.88
AirDet [16]	✗	5.97	10.52	5.98	6.58	12.02	6.33	7.00	12.95	6.71	7.76	14.28	7.31
FS-DETR [52]	✗	7.00	13.60	7.50	8.90	17.50	9.00	10.00	18.80	10.00	10.90	20.70	10.80
TIDE [3]	✗	9.87	16.16	10.51	12.55	21.29	13.26	12.35	20.81	12.73	11.75	19.89	12.01
PFN	✗	14.41	29.29	12.60	15.03	30.59	13.40	14.92	30.78	13.08	14.91	31.08	12.54
FRCN [53]	✓	3.26	6.66	3.04	3.73	7.79	3.22	4.59	9.52	4.07	5.32	11.20	4.54
TFA _{fc} [24]	✓	2.78	5.39	2.36	4.14	7.98	4.01	6.33	12.10	5.94	7.92	15.58	7.29
TFA _{cos} [24]	✓	3.09	5.24	3.21	4.21	7.70	4.35	6.05	11.48	5.93	7.61	14.56	7.17
FSDetView [26]	✓	2.20	6.20	0.90	3.40	10.00	1.50	5.20	14.70	2.10	8.20	21.60	4.70
MPSR [30]	✓	3.34	6.11	3.25	5.41	9.68	5.52	5.70	10.54	5.50	7.20	13.55	6.89
A-RPN [27]	✓	4.59	8.85	4.37	6.15	12.05	5.76	8.24	15.52	7.92	9.02	17.29	8.53
W.Zhang et al. [14]	✓	4.40	7.50	4.90	5.60	9.90	5.90	7.20	13.30	7.40	—	—	—
FADI [9]	✓	5.70	10.40	6.00	7.00	13.01	7.00	8.60	15.80	8.30	10.10	18.60	11.90
AirDet [16]	✓	6.10	11.40	6.04	8.73	16.24	8.35	9.95	19.39	9.09	10.81	20.75	10.27
Meta-DETR [6]	✓	7.50	12.50	7.70	—	—	—	13.50	21.70	14.00	15.40	25.00	15.80

TABLE V
IMPACT OF PROTOTYPE-UPDATE ORDERING ON
COCO NOVEL SET (AP@50)

Ordering Strategy	1-shot	2-shot	3-shot	5-shot
Ascending (easy → hard)	29.21	30.47	30.65	30.96
Descending (hard → easy)	29.08	30.32	30.51	30.87
Class-balanced round-robin	29.29	30.59	30.78	31.08
Random shuffling (mean ± std)	29.17 ±0.06	30.41 ±0.08	30.63 ±0.07	30.94 ±0.05

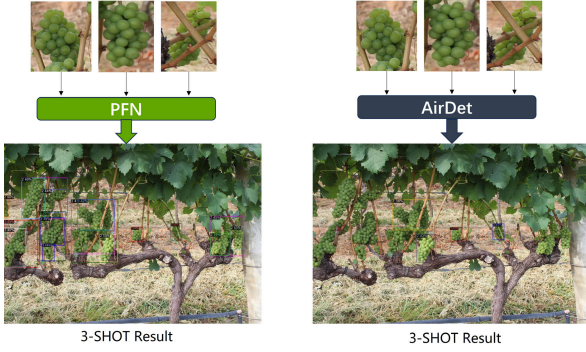


Fig. 8. Qualitative comparison on Embrapa Wine Grape dataset [57] (PFN versus AirDet): Our method achieves superior localization accuracy with fewer missed detections.

and Playing Card Detection dataset. We also compare our method visually with AirDet using the Embrapa Wine Grape dataset [57] as shown in Fig. 8. These datasets represent distinct visual domains, including agricultural scenes, industrial safety equipment, and recreational objects.

Table XII presents the cross-domain evaluation results. Our PFN demonstrates consistent performance across these diverse domains, showcasing its robust generalization capabilities. The Global Wheat Detection dataset, with its agricultural imagery and varying lighting conditions, poses unique challenges that our dynamic prototype fusion effectively addresses. Similarly, the Pictor-PPE dataset's industrial setting and the Playing

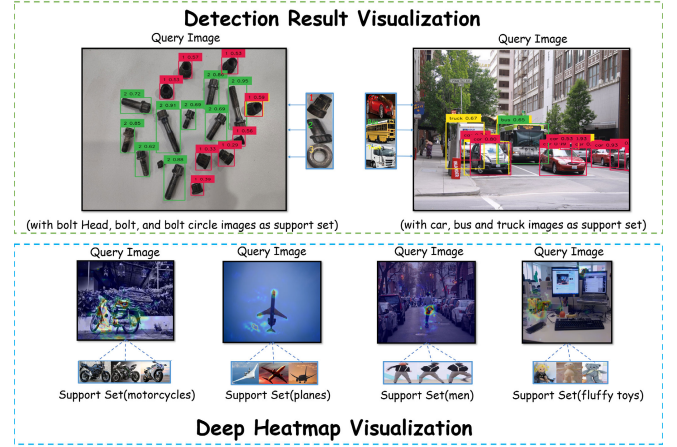


Fig. 9. Detection and deep visualization result from the proposed PFN. The images above demonstrate detection results of different types of screws and cars, while the images below depict the model's main focus on novel objects' representative parts in detection.

Card dataset's controlled indoor environment validate PFN's adaptability across different visual contexts.

D. Ablation Study and Visualization

To better understand the sensitivity of the proposed PFN, we conduct an extended ablation study on the MSCOCO 1–30 few-shot dataset to assess the impact of each major component. As shown in Table VII, we compare the performance of PFN with and without the following components: 1) BMCA; 2) feature pyramid; 3) support image segmentation; and 4) SA-OQS. The results demonstrate that each component contributes to the overall performance of PFN. Specifically, the BMCA-based encoder fusion, SA-OQS, and segmentation sampling strategy are crucial for improving the performance and query-support relations of PFN, while the feature pyramid also plays an important role in enhancing the multiscale information aggregation ability of PFN.

During training, BMCA aligns hierarchical support/query features and speeds up convergence; the feature pyramid

TABLE VI
FSOD RESULTS ON THE PASCAL VOC CROSS-DOMAIN DATASET

Shots		1			2			3			5		
Method	Finetune	AP	AP_{50}	AP_{75}	AP	AP_{50}	AP_{75}	AP	AP_{50}	AP_{75}	AP	AP_{50}	AP_{75}
A-RPN [27]	✗	10.45	18.10	10.32	13.10	22.60	13.17	14.05	24.08	5.05	15.87	35.03	15.26
AirDet [16]	✗	11.92	21.33	11.56	15.80	26.80	16.08	16.08	28.61	6.71	17.83	29.78	18.38
TIDE [3]	✗	16.88	26.81	17.49	19.67	33.02	20.17	20.90	35.15	21.36	21.36	37.06	21.31
PFN	✗	19.84	35.12	19.43	20.67	37.36	21.92	22.83	38.65	22.55	24.18	41.36	23.34
FRCN [53]	✓	4.49	9.44	3.85	5.20	11.92	3.84	6.50	14.39	5.11	6.55	14.48	5.09
TFA _{cos} [24]	✓	4.66	7.97	5.14	6.59	11.91	6.49	8.78	17.09	8.15	10.46	20.93	9.53
TFA _{fc} [24]	✓	4.40	8.60	4.21	7.02	13.80	6.21	9.24	18.48	8.03	11.11	22.83	9.78
FSDetView [26]	✓	4.80	14.10	1.40	3.70	11.60	0.60	6.60	22.00	1.20	10.80	26.50	5.50
MPSR [30]	✓	6.01	11.23	5.74	8.20	15.08	8.22	10.08	18.29	9.99	11.49	21.33	12.06
A-RPN [27]	✓	9.49	17.41	9.42	12.71	23.66	12.44	14.89	26.30	14.76	15.09	28.08	14.17
AirDet [16]	✓	13.33	24.64	12.68	17.51	30.35	17.61	17.68	32.05	17.34	18.27	33.02	17.69

TABLE VII
ABLATION STUDY ON MAJOR COMPONENTS FOR EVALUATING PFN

Methods	COCO Novel Set (AP50)					
	1	2	3	5	10	30
PFN	29.29	30.59	30.78	31.08	31.17	31.09
PFN w/o BMCA	26.18	27.12	28.41	28.68	29.12	29.13
PFN w/o Feature Pyramid	26.25	26.80	26.96	27.43	27.06	27.46
PFN w/o Segmentation	22.58	25.32	25.37	26.58	27.70	28.23
PFN w/o Support-Aware Selection	23.52	24.34	25.33	27.55	28.71	26.23

TABLE VIII
ROBUSTNESS OF PFN TO GAUSSIAN NOISE ON THE COCO NOVEL SET (AP@50)

Noise Level (σ)	Shots			
	1	2	3	5
0 (clean)	29.29	30.59	30.78	31.08
1	28.85	30.12	30.32	30.61
5	27.73	29.04	29.26	29.62
10	26.48	27.91	28.18	28.49

TABLE IX
ABLATION STUDY ON DIFFERENT FEATURE STAGE COMBINATIONS (COCO NOVEL SET, AP@50)

Stage Combination	Shots			
	1	2	3	5
S2-S3	24.82	25.91	26.18	26.87
S3-S4	27.59	28.73	28.94	29.26
S4-S5	26.48	27.62	27.83	28.21
S2-S5 (Ours)	29.29	30.59	30.78	31.08

TABLE X
COMPARISON OF DIFFERENT FLATTENING STRATEGIES (COCO NOVEL SET, AP@50)

Flattening Strategy	Shots			
	1	2	3	5
Channel-wise Flattening	26.18	27.09	27.34	27.93
Hierarchical Pooling	26.94	27.81	28.05	28.64
Patch-based Flattening	28.07	28.95	29.18	29.73
Spatial Flattening (Ours)	29.29	30.59	30.78	31.08

injects rich spatial priors that benefit objects of diverse scales; segmentation masks filter background noise to yield cleaner prototypes; and SA-OQS dynamically prunes low-relevance queries, producing stabler gradients and ~15% faster infer-

TABLE XI
EFFECTIVENESS OF DIFFERENT POSITION ENCODING METHODS (COCO NOVEL SET, AP@50)

Position Encoding	Shots			
	1	2	3	5
No Position Encoding	24.56	25.73	25.91	26.48
Learnable Position Encoding	27.41	28.52	28.71	29.14
Relative Position Encoding	27.93	29.05	29.21	29.67
Sinusoidal (Ours)	29.29	30.59	30.78	31.08

TABLE XII
EXTENDED CROSS-DOMAIN EVALUATION RESULTS (AP@50) ON DIVERSE DATASETS

Method	Dataset	Shots		
		1	3	5
AirDet	Global Wheat Detection	18.34	20.67	22.18
	Pictor-PPE	16.89	19.24	21.41
	Playing Card Detection	11.73	12.92	13.84
PFN (Ours)	Global Wheat Detection	23.87	25.94	27.63
	Pictor-PPE	21.52	24.18	26.34
	Playing Card Detection	19.26	20.45	20.92

ence. Ablating any single module incurs a distinct 2–4 pp drop in AP_{50} , confirming their complementary roles during both training and inference.

1) *Comprehensive Parameter Analysis*: To provide more thorough justification for our parameter choices, we conduct comprehensive ablation studies on the key components of BMCA and multiscale feature fusion.

Stage Selection Analysis: Table IX presents the performance comparison of different feature stage combinations. Our choice of S2-S5 stages achieves optimal performance by balancing semantic richness and spatial resolution. Using only early stages (S2-S3) loses high-level semantic information, while relying solely on late stages (S4-S5) sacrifices fine-grained spatial details.

Flattening Strategy Comparison: As shown in Table X, we evaluate different feature flattening approaches. Our spatial flattening strategy outperforms alternative methods, including channel-wise flattening and hierarchical pooling,

TABLE XIII
ANALYSIS OF DIFFERENT MODULE COMBINATIONS
(COCO NOVEL SET, AP@50)

Modules				Shots (AP@50)			
BMCA	FP	Seg	SA-OQS	1	2	3	5
✓				19.23	20.47	20.83	21.69
	✓			24.05	25.34	25.72	26.53
		✓		21.68	22.91	23.24	24.08
			✓	21.14	22.39	22.73	23.56
				22.49	23.78	24.15	24.94
✓	✓			27.12	28.27	28.56	29.18
✓		✓		26.01	27.19	27.47	28.14
✓			✓	27.58	28.71	28.99	29.64
	✓	✓		24.91	26.08	26.35	27.02
✓	✓	✓	✓	29.29	30.59	30.78	31.08

TABLE XIV
TIME AND SPACE COMPLEXITY OF THE PROPOSED PFN

Metric	PFN
Parameters	100 MB
Average inference time [†]	718 ms
FLOPs / image	492 G
Peak GPU memory	≈4 GB

[†]Inference time is the mean of the individual runtimes measured for 1-, 2-, 3-, 5-, and 10-shot settings.

providing better spatial-semantic alignment for cross-attention mechanisms.

Position Encoding Methods: Table XI demonstrates the effectiveness of different position encoding techniques. Sinusoidal position encoding achieves the best performance, providing stable positional information across different input resolutions compared to learnable and relative position encodings.

2) Module Combination Analysis: Table XIII presents a comprehensive analysis of different module combinations. The results demonstrate that each module contributes uniquely to the overall performance, with the full PFN achieving the best results. Notably, the combination of BMCA and feature pyramid provides substantial improvements, while adding segmentation and SA-OQS further enhances performance across different shot settings.

Visualization of the detection result and deep fused features of PFN is shown in Fig. 9 as a vivid showcase for the advantages of PFN. It is capable of detecting novel objects with few support images without fine-tuning again while sustaining decent inference speed, as shown in Table XIV. Thanks to SA-OQS, predicted boxes concentrate on salient targets while suppressing distractors, echoing the quantitative gains. Remarkably, for objects that are hard to describe via language text prompt [58], our method provides a more flexible option of real-time visual prompt detection.

E. Feature-Space Visualization (t-SNE)

To illustrate how PFN organizes visual concepts, we project the 768-D token embeddings from the penultimate PFN block

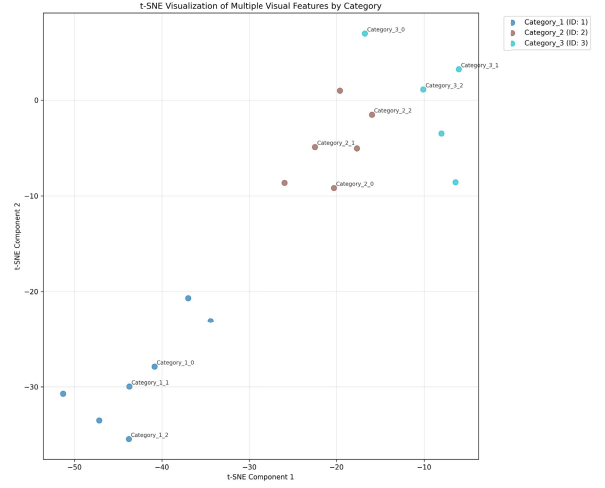


Fig. 10. t-SNE projection of 768-D features for three novel classes (Class 1: 5 shots, Class 2: 6 shots, Class 3: 5 shots).

TABLE XV
PER-MODULE COMPUTATIONAL COST BREAKDOWN OF PFN

Module	FLOPs (G)	Share (%)	Memory (GB)
Swin-T Backbone (frozen)	88.4	18.0	0.8
Multi-Scale FPN[†]	72.3	14.7	0.6
<i>Transformer Encoder:</i>			
BMCA (cross-attn)[†]	24.8	5.0	0.2
MDSA (self-attn)	215.8	43.9	1.2
FFN	46.8	9.5	0.5
Cross-Relation Decoder	35.2	7.2	0.4
Prediction Heads	8.7	1.8	0.1
Total	492.0	100.0	≈3.8

[†]Proposed modules: combined overhead is only 19.7%.

to 2-D with t-SNE(perplexity = 30). As shown in Fig. 10, samples from the same novel category form tight, compact clusters while remaining well-separated across categories, confirming that PFN learns both intraclass coherence and interclass discrimination even under few-shot supervision.

F. Noise-Robustness Experiment

Besides component ablations, we also examine how well PFN withstands input corruption. We inject zero-mean Gaussian noise with four standard deviation levels ($\sigma = 0, 1, 5, 10$ on the 0–255 pixel scale) into *both* support and query images and reevaluate on the COCO novel set. Table VIII shows that PFN’s AP@50 drops by less than three points even under the heaviest noise, confirming that the transformer-based fusion and attention mechanisms effectively suppress random perturbations.

G. Computational Complexity and Efficiency

1) Resource Footprint: The complete model occupies ~100 MB on disk and yields an average inference latency of 718 ms per image (mean over 1-, 2-, 3-, 5- and 10-shot runs), on a single A800 GPU with batch size 1. Profiling results

TABLE XVI

PER-NOVEL-CLASS COMPUTATIONAL OVERHEAD AFTER SUPPORT SHOTS ARE PROVIDED

Method	Extra epochs / iter.	GPU h [†]	Add. cost / shot
TFA _{fc} [24]	10 epochs	0.45	–
MPSR [30]	15 epochs	0.62	–
AirDet [16]	10k it. (8 epochs)	0.51	–
TIDE [3]	None	0.00	–
PFN (ours)	None	0.00	15 ms (mask)

[†]Wall-clock hours on one A100 (40 GB) for the post-support procedure only.

with *fvcore* [59] indicate that our proposed modules are lightweight: the multiscale FPN contributes 14.7% (72.3 GFLOPs), and the dual-branch BMCA adds only 5.0% (24.8 GFLOPs) to the total budget; combined, they incur just 19.7% computational overhead. Crucially, these modules benefit from efficient parallelization on GPUs, which have almost no major influence on the actual inference time. As shown in Table XV, the majority of the cost comes from standard backbone and self-attention operations.

2) *Computational Complexity of BMCA*: Since BMCA operates between query tokens ($L \approx 5000$) and only ~ 20 prototypes, the cross-attention cost is minimal—only 24.8 GFLOPs (5.0% of total). The bulk of the encoder cost comes from MDSA self-attention (43.9%), not from our proposed dual-branch module. Table XV summarizes the per-module breakdown.

A key promise of PFN is that *no detector fine-tuning* is needed once the K support images arrive; the only extra work is the lightweight mask preprocessing described in Section III-B2. Table XVI benchmarks that overhead against popular few-shot pipelines on a single NVIDIA A100 GPU. Even with the segmentation step, PFN adds merely ≈ 15 ms per support image—orders of magnitude cheaper than the 0.4–0.6 GPU-hour fine-tuning phase of TFA, MPSR, or AirDet.

V. FUTURE WORK

Note that there are two limitations in our PFN, i.e., dependence on segmentation quality and the frozen support backbone mechanism. To further advance test-time FSOD, future research will focus on several promising directions as follows.

- 1) *Advanced Prototype Modeling*: Investigate sophisticated methods for constructing prototypes that accurately represent intraclass variance from support samples, and develop adaptive background prototypes to enhance robustness across diverse scenes.
- 2) *Reduced Supervision and Noise Robustness*: Prioritize techniques to reduce dependence on precise segmentation masks, concurrently enhancing inherent robustness to various image noise types to ensure practicality in real-world scenarios.
- 3) *Enhanced Generalization*: Focus efforts on improving generalization capabilities to novel object categories and

domains significantly different from the training data, developing adaptive methods for unfamiliar contexts.

VI. CONCLUSION

We develop a novel dynamic PFN for test-time FSOD, which will address two key problems: domain/category shift and limited sample sizes. To tackle the test-time scenario, in PFN, only the prototype is updated during the configuration. Moreover, a dynamic prototype fusion strategy is incorporated to alleviate the negative transfer caused by the domain shift. Then, to uncover the issues of limited sample sizes, PFN offers dual-level multiscale information integration and mask processing with support images for thorough exploitation of information within the support images. Empirical evidence demonstrates that our PFN significantly improves upon existing methods, achieving substantial performance gains and establishing new state-of-the-art.

ACKNOWLEDGMENT

The authors would like to thank Dr. Huayang Tang, Dr. Miaolong Ye, Dr. Meng Tan, and Dr. Lei Li for the proofreading of this manuscript and for their technical support.

REFERENCES

- [1] M. Köhler, M. Eisenbach, and H.-M. Gross, “Few-shot object detection: A comprehensive survey,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 11958–11978, Sep. 2024.
- [2] G. Huang, I. Laradji, D. Vázquez, S. Lacoste-Julien, and P. Rodríguez, “A survey of self-supervised and few-shot object detection,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 4, pp. 4071–4089, Apr. 2023.
- [3] W. Li et al., “TIDE: Test-time few-shot object detection,” *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 54, no. 11, pp. 6500–6509, Nov. 2024.
- [4] L. Liu, B. Ma, Y. Zhang, X. Yi, and H. Li, “AFD-Net: Adaptive fully-dual network for few-shot object detection,” in *Proc. 29th ACM Int. Conf. Multimedia*, Oct. 2021, pp. 2549–2557.
- [5] W. Yin, “Meta-learning for few-shot natural language processing: A survey,” 2020, *arXiv:2007.09604*.
- [6] G. Zhang, Z. Luo, K. Cui, and S. Lu, “Meta-DETR: Few-shot object detection via unified image-level meta-learning,” 2021, *arXiv:2103.11731*.
- [7] X. Wu, D. Sahoo, and S. Hoi, “Meta-RCNN: Meta learning for few-shot object detection,” in *Proc. 28th ACM Int. Conf. Multimedia*, Oct. 2020, pp. 1679–1687.
- [8] L. Liu et al., “GenDet: Meta learning to generate detectors from few shots,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 8, pp. 3448–3460, Aug. 2022.
- [9] H. Chen, Y. Wang, G. Wang, and Y. Qiao, “LSTD: A low-shot transfer detector for object detection,” in *Proc. AAAI Conf. Artif. Intell.*, 2018, vol. 32, no. 1, pp. 2836–2843.
- [10] B. Li, Q. Yao, and K. Wang, “A review on vision-based pedestrian detection in intelligent transportation systems,” in *Proc. 9th IEEE Int. Conf. Netw., Sens. Control*, Apr. 2012, pp. 393–398.
- [11] Z. Yang, Y. Wang, X. Chen, J. Liu, and Y. Qiao, “Context-transformer: Tackling object confusion for few-shot detection,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, 2020, pp. 12653–12660.
- [12] Z. Zhao, Q. Liu, and Y. Wang, “Exploring effective knowledge transfer for few-shot object detection,” in *Proc. 30th ACM Int. Conf. Multimedia*, Oct. 2022, pp. 6831–6839.
- [13] Z. Peng, Z. Li, J. Zhang, Y. Li, G.-J. Qi, and J. Tang, “Few-shot image recognition with knowledge transfer,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 441–449.
- [14] W. Zhang and Y.-X. Wang, “Hallucination improves few-shot object detection,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 13008–13017.
- [15] B. Demirel, O. B. Baran, and R. G. Cinbis, “Meta-tuning loss functions and data augmentation for few-shot object detection,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 7339–7349.

- [16] B. Li, C. Wang, P. Reddy, S. Kim, and S. Scherer, "AirDet: Few-shot detection without fine-tuning for autonomous exploration," in *Proc. 17th Eur. Conf. Comput. Vis.*, Oct. 2022, pp. 427–444.
- [17] X. Zhou, W. Liang, S. Shimizu, J. Ma, and Q. Jin, "Siamese neural network based few-shot learning for anomaly detection in industrial cyber-physical systems," *IEEE Trans. Ind. Inform.*, vol. 17, no. 8, pp. 5790–5798, Aug. 2021.
- [18] R. Theagarajan and B. Bhanu, "Privacy preserving defense for black box classifiers against on-line adversarial attacks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 12, pp. 9503–9520, Dec. 2022.
- [19] Y. Liu, M.-M. Cheng, X.-Y. Zhang, G.-Y. Nie, and M. Wang, "DNA: Deeply supervised nonlinear aggregation for salient object detection," *IEEE Trans. Cybern.*, vol. 52, no. 7, pp. 6131–6142, Jul. 2022.
- [20] G. Li, Z. Liu, D. Zeng, W. Lin, and H. Ling, "Adjacent context coordination network for salient object detection in optical remote sensing images," *IEEE Trans. Cybern.*, vol. 53, no. 1, pp. 526–538, Jan. 2023.
- [21] W. Li and S. Chen, "Unsupervised domain adaptation with progressive adaptation of subspaces," *Pattern Recognit.*, vol. 132, Dec. 2022, Art. no. 108918.
- [22] H. Wang, W. Yang, J. Wang, R. Wang, L. Lan, and M. Geng, "Pairwise similarity regularization for adversarial domain adaptation," in *Proc. 28th ACM Int. Conf. Multimedia*, Oct. 2020, pp. 2409–2418.
- [23] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. ECCV*, Zurich, Switzerland, 2014, pp. 740–755.
- [24] X. Wang, T. S. Huang, J. E. Gonzalez, T. Darrell, and F. Yu, "Frustratingly simple few-shot object detection," in *Proc. Int. Conf. Mach. Learn.*, vol. 1, 2020, pp. 9919–9928.
- [25] X. Yan, Z. Chen, A. Xu, X. Wang, X. Liang, and L. Lin, "Meta R-CNN: Towards general solver for instance-level low-shot learning," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 9576–9585.
- [26] Y. Xiao, V. Lepetit, and R. Marlet, "Few-shot object detection and viewpoint estimation for objects in the wild," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 3090–3106, Mar. 2023.
- [27] Q. Fan, W. Zhuo, C.-K. Tang, and Y.-W. Tai, "Few-shot object detection with attention-RPN and multi-relation detector," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 4013–4022.
- [28] Y. Lu, X. Chen, Z. Wu, and J. Yu, "Decoupled metric network for single-stage few-shot object detection," *IEEE Trans. Cybern.*, vol. 53, no. 1, pp. 514–525, Jan. 2023.
- [29] C. Chen, X. Yang, J. Zhang, B. Dong, and C. Xu, "Category knowledge-guided parameter calibration for few-shot object detection," *IEEE Trans. Image Process.*, vol. 32, pp. 1092–1107, 2023.
- [30] J. Wu, S. Liu, D. Huang, and Y. Wang, "Multi-scale positive sample refinement for few-shot object detection," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Oct. 2020, pp. 456–472.
- [31] C. Zhu, F. Chen, U. Ahmed, Z. Shen, and M. Savvides, "Semantic relation reasoning for shot-stable few-shot object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Mar. 2021, pp. 8782–8791.
- [32] Z. Yang, M. Shi, C. Xu, V. Ferrari, and Y. Avrithis, "Training object detectors from few weakly-labeled and many unlabeled images," *Pattern Recognit.*, vol. 120, Dec. 2021, Art. no. 108164.
- [33] G. Kim, H.-G. Jung, and S.-W. Lee, "Few-shot object detection via knowledge transfer," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, Oct. 2020, pp. 3564–3569.
- [34] Y.-X. Wang, R. Girshick, M. Hebert, and B. Hariharan, "Low-shot learning from imaginary data," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7278–7286.
- [35] A. Bansal, K. Sikka, G. Sharma, R. Chellappa, and A. Divakaran, "Zero-shot object detection," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 384–400.
- [36] G. Han, J. Ma, S. Huang, L. Chen, and S.-F. Chang, "Few-shot object detection with fully cross-transformer," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 5321–5330.
- [37] M. Caron et al., "Emerging properties in self-supervised vision transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 9650–9660.
- [38] T. Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jul. 2017, pp. 2117–2125.
- [39] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable DETR: Deformable transformers for end-to-end object detection," 2020, *arXiv:2010.04159*.
- [40] J. Li, Z. Pan, Q. Liu, Y. Cui, and Y. Sun, "Complementarity-aware attention network for salient object detection," *IEEE Trans. Cybern.*, vol. 52, no. 2, pp. 873–886, Feb. 2022.
- [41] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 6000–6010.
- [42] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 213–229.
- [43] A. Kirillov et al., "Segment anything," 2023, *arXiv:2304.02643*.
- [44] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2961–2969.
- [45] C. Rother, V. Kolmogorov, and A. Blake, "Grabcut: Interactive foreground extraction using iterated graph cuts," *ACM Trans. Graph.*, vol. 23, no. 3, pp. 309–314, 2004.
- [46] Y. Wang, X. Zhang, T. Yang, and J. Sun, "Anchor DETR: Query design for transformer-based detector," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 3, Jun. 2022, pp. 2567–2575.
- [47] S. Liu et al., "DAB-DETR: Dynamic anchor boxes are better queries for DETR," 2022, *arXiv:2201.12329*.
- [48] Z. Yao, J. Ai, B. Li, and C. Zhang, "Efficient DETR: Improving end-to-end object detector with dense prior," 2021, *arXiv:2104.01318*.
- [49] H. Rezaatofghi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid, and S. Savarese, "Generalized intersection over union: A metric and a loss for bounding box regression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 658–666.
- [50] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2009, pp. 248–255.
- [51] M. Oquab et al., "DINOv2: Learning robust visual features without supervision," 2023, *arXiv:2304.07193*.
- [52] A. Bulat, R. Guerrero, B. Martinez, and G. Tzimiropoulos, "FS-DETR: Few-shot detection transformer with prompting and without re-training," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 11759–11768.
- [53] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 39, 2016, pp. 1137–1149.
- [54] E. David et al., "Global wheat head detection (GWHD) dataset: A large and diverse dataset of high-resolution RGB-labelled images to develop and benchmark wheat head detection methods," *Plant Phenomics*, vol. 2020, Apr. 2020, Art. no. 3521852.
- [55] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The Pascal visual object classes (VOC) challenge," *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, Jun. 2010.
- [56] N. D. Nath, A. H. Behzadan, and S. G. Paal, "Deep learning for site safety: Real-time detection of personal protective equipment," *Autom. Construct.*, vol. 112, Apr. 2020, Art. no. 103085.
- [57] T. T. Santos, L. L. de Souza, A. A. dos Santos, and S. Avila, "Grape detection, segmentation, and tracking using deep neural networks and three-dimensional association," *Comput. Electron. Agricult.*, vol. 170, Mar. 2020, Art. no. 105247.
- [58] L. H. Li et al., "Grounded language-image pre-training," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 10965–10975.
- [59] Meta AI Research.(2019). *FVcore: A Lightweight Core Library for Computer Vision*. Accessed: Dec. 10, 2025. [Online]. Available: <https://github.com/facebookresearch/fvcore>



Yanlai Wu is currently pursuing the M.S. degree in electrical and computer engineering with Rowan University, Glassboro, NJ, USA.

He is a Research Assistant at Shandong Jianzhu University, Jinan, China. His current research involves pattern recognition, computer vision, and natural language processing.



Yuan Li received the master's degree from the College of Engineering, Arizona State University, Tempe, AZ, USA, in 2022, with and possesses over ten years of experience in computer vision and video processing algorithm engineering.

He previously worked at Sony, focusing on the research and development of video-related algorithms. Currently, he leads the AI Algorithm Engineering Team at Intellicloud's Product and Engineering Research Center, dedicated to the practical application of AI algorithms.



Weikai Li (Associate Member, IEEE) received the B.S. degree in information and computing science and the M.S. degree in computer science and technique from Chongqing Jiaotong University, Chongqing, China, in 2015 and 2018, respectively, and the Ph.D. degree from the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2022.

He is now a Professor at the School of Computer and Artificial Intelligence, Shandong Jianzhu University, Jinan, China. His research interests include pattern recognition and machine learning.



Hongfeng (Jason) Wei is currently a Visiting Professor at the University of California, Los Angeles, Los Angeles, CA, USA, and the University of California Irvine, Irvine, CA. He is the CEO with China Science IntelliCloud Technology, China. He is also a Senior Consultant for Pfizer Pharmaceuticals, Amazon, and General Electric and as the CTO of Nasdaq listed Technology Company.



Ying (Gina) Tang (Fellow, IEEE) received the B.S. and M.S. degrees from the Northeastern University, Shenyang, P.R. China, in 1996 and 1998, respectively, and Ph. D degree from New Jersey Institute of Technology, Newark, NJ, USA, in 2001.

She is currently a Professor with the Department of Electrical and Computer Engineering and Associate Director of the Machine Learning and Artificial Intelligence Virtual Reality Center (MAVRC) at Rowan University, Glassboro, NJ, USA. Her current research interests are in cyber-physical social systems, exploring the intersection of artificial intelligence and extended reality to improve adaptation and personalization in complex systems. Her work has resulted in three USA patents, and over 260 peer-reviewed publications, including 93 journal articles, two edited books, and six book / encyclopedia chapters.

Dr. Tang is presently a Senior Editor of TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS and IEEE TRANSACTIONS ON INTELLIGENT VEHICLES, and an Associate Editor of TRANSACTIONS ON COMPUTATIONAL SOCIAL SYSTEMS and *Springer's Discover Artificial Intelligence*. She is the Founding Chair of Technical Committee on Intelligent Solutions to Human-aware Sustainability for IEEE Systems, Man, & Cybernetic, and the Founding Chair of Technical Committee on Sustainable Production Automation for IEEE Robotic and Automation.